AI Solution: Object Counting using Segment Anything Model (SAM)

Author: Sanjay
Task: Count items in images using AI techniques
Target Accuracy: > 95%

Approach

This solution uses Meta's **Segment Anything Model (SAM)** for zero-shot object segmentation:

- Automatic mask generation for all objects
- Area-based filtering (median-based adaptive thresholding)
- IoU-based duplicate removal
- Containment analysis for nested objects

Model

- **SAM ViT-B**: Base model (~375 MB)
- Pre-trained on SA-1B dataset (11M images, 1.1B masks)
- Zero-shot generalization to new object types

1. Installation

In [61]:
```
# Install required packages
!pip install segment-anything opencv-python matplotlib torch torchvision
```

```
zsh:1: command not found: pip
```

2. Download SAM Checkpoint

In [62]:
```
# Download SAM ViT-B checkpoint (~375 MB)
!wget https://dl.fbaipublicfiles.com/segment_anything/sam_vit_b_01ec64.pt
```

```
zsh:1: command not found: wget
```

3. Import Libraries

In [63]:
```
import torch
import cv2
import numpy as np
import matplotlib.pyplot as plt
from segment_anything import sam_model_registry, SamAutomaticMaskGenerato
from pathlib import Path
```

```
# For Colab file upload
try:
    from google.colab import files
    IN_COLAB = True
except ImportError:
    IN_COLAB = False

print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
```

```
PyTorch version: 2.11.0
CUDA available: False
```

4. Load SAM Model

In [64]:
```
# Model configuration
sam_checkpoint = "sam_vit_b_01ec64.pth"
model_type = "vit_b"
device = "cuda" if torch.cuda.is_available() else "cpu"

print(f"Loading SAM model on {device}...")

# Load SAM
sam = sam_model_registry[model_type](checkpoint=sam_checkpoint)
sam.to(device)

# Initialize mask generator
mask_generator = SamAutomaticMaskGenerator(sam)

print("✓ Model loaded successfully")
```

```
Loading SAM model on cpu...
✓ Model loaded successfully
```

5. Define Helper Functions

In [65]:
```
def compute_iou(mask1, mask2):
    """Compute Intersection over Union between two masks."""
    intersection = np.logical_and(mask1, mask2).sum()
    union = np.logical_or(mask1, mask2).sum()
    return intersection / (union + 1e-8)

def compute_containment(mask_small, mask_large):
    """Compute percentage of small mask contained in large mask."""
    intersection = np.logical_and(mask_small, mask_large).sum()
    return intersection / (mask_small.sum() + 1e-8)

def filter_masks_by_area(masks, min_mult=0.3, max_mult=2.0):
    """Filter masks based on median area."""
    if not masks:
        return []

    areas = np.array([m['area'] for m in masks])
    median_area = np.median(areas)

    min_area = min_mult * median_area
    max_area = max_mult * median_area
```

```
    filtered = [m for m in masks if min_area < m['area'] < max_area]

    print(f"Area filtering: {len(masks)} -> {len(filtered)} masks")
    print(f"Median area: {median_area:.0f}, Range: [{min_area:.0f}, {max_

    return filtered

def remove_duplicates(masks, iou_threshold=0.6, containment_threshold=0.8
    """Remove overlapping/duplicate masks."""
    if not masks:
        return []

    # Sort by area (largest first)
    sorted_masks = sorted(masks, key=lambda x: x['area'], reverse=True)
    final_masks = []

    for mask in sorted_masks:
        keep = True

        for existing in final_masks:
            # Check IoU
            iou = compute_iou(mask['segmentation'], existing['segmentatio
            if iou > iou_threshold:
                keep = False
                break

            # Check containment
            containment = compute_containment(
                mask['segmentation'],
                existing['segmentation']
            )
            if containment > containment_threshold:
                keep = False
                break

        if keep:
            final_masks.append(mask)

    print(f"Duplicate removal: {len(sorted_masks)} -> {len(final_masks)}

    return final_masks
```

6. Object Detection Pipeline

In [66]:

```
def detect_and_count(image, min_area_mult=0.3, max_area_mult=2.0,
                     iou_threshold=0.6, containment_threshold=0.8):
    """
    Detect and count objects in image using SAM.

    Args:
        image: Input BGR image
        min_area_mult: Minimum area multiplier for filtering
        max_area_mult: Maximum area multiplier for filtering
        iou_threshold: IoU threshold for duplicate removal
        containment_threshold: Containment threshold for nested objects

    Returns:
        overlay: Annotated image
```

```
        count: Object count
        final_masks: List of mask dictionaries
    """
    # Convert BGR to RGB
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Generate masks
    print("Generating masks...")
    raw_masks = mask_generator.generate(image_rgb)
    print(f"Raw masks detected: {len(raw_masks)}")

    # Filter by area
    filtered_masks = filter_masks_by_area(raw_masks, min_area_mult, max_a

    # Remove duplicates
    final_masks = remove_duplicates(filtered_masks, iou_threshold, contai

    # Create overlay visualization
    overlay = image_rgb.copy()

    for mask in final_masks:
        seg = mask['segmentation']
        color = np.random.randint(0, 255, 3)
        overlay[seg] = overlay[seg] * 0.5 + color * 0.5

    count = len(final_masks)
    print(f"\n✓ Final count: {count} objects\n")

    return overlay.astype(np.uint8), count, final_masks
```

7. Load and Process Images

In [67]:
```
# Upload images (for Colab) or specify paths
if IN_COLAB:
    uploaded = files.upload()
    image_files = list(uploaded.keys())
else:
    # Example image paths
    image_files = [
        '/Users/sanju/Desktop/Sanjay/Screws_2024_07_15/img1_43_nosy.jpg',
    ]

print(f"Images to process: {len(image_files)}")
```

```
Images to process: 1
```

8. Run Detection

In [68]:
```
# Process each image
results = []

for fname in image_files:
    print(f"\n{'='*60}")
    print(f"Processing: {fname}")
    print('='*60)

    # Load image
```

```
    image = cv2.imread(fname)

    if image is None:
        print(f"Could not read {fname}")
        continue

    # Detect objects
    overlay, count, masks = detect_and_count(
        image,
        min_area_mult=0.3,
        max_area_mult=2.0,
        iou_threshold=0.6,
        containment_threshold=0.8
    )

    results.append({
        'filename': fname,
        'count': count
    })

    # Display results
    plt.figure(figsize=(12, 8))
    plt.imshow(overlay)
    plt.axis('off')
    plt.title(f"{fname} - Detected: {count} objects", fontsize=14, fontwe
    plt.tight_layout()
    plt.show()
```

```
============================================================
Processing: /Users/sanju/Desktop/Sanjay/Screws_2024_07_15/img1_43_nosy.jpg
============================================================
Generating masks...
Raw masks detected: 51
Area filtering: 51 -> 50 masks
Median area: 1592, Range: [478, 3184]
Duplicate removal: 50 -> 42 masks

✓ Final count: 42 objects
```

/Users/sanju/Desktop/Sanjay/Screws_2024_07_15/img1_43_nosy.jpg - Detected: 42 objects



9. Accuracy Calculation

In [69]:
```
# Example: Calculate accuracy if ground truth is known
# Replace with your actual ground truth values

ground_truth = {
    '/Users/sanju/Desktop/Sanjay/Screws_2024_07_15/img1_43_nosy.jpg': 43,
}

print("\n=== Accuracy Report ===")
print(f"{'Image':<30} {'Predicted':<12} {'Ground Truth':<15} {'Accuracy'}
print("-" * 70)

total_accuracy = 0
count = 0

for result in results:
    fname = result['filename']
    predicted = result['count']

    if fname in ground_truth:
        gt = ground_truth[fname]
        accuracy = 1 - abs(predicted - gt) / gt
        accuracy_pct = accuracy * 100

        print(f"{fname:<30} {predicted:<12} {gt:<15} {accuracy_pct:.2f}%"

        total_accuracy += accuracy
        count += 1

if count > 0:
    avg_accuracy = (total_accuracy / count) * 100
    print("-" * 70)
```

```
    print(f"Average Accuracy: {avg_accuracy:.2f}%")
    print(f"Target: > 95.00%")
    print(f"Status: {'✓ PASSED' if avg_accuracy > 95 else '✗ FAILED'}")
```

```
=== Accuracy Report ===
Image                          Predicted    Ground Truth    Accuracy
------------------------------------------------------------------------
/Users/sanju/Desktop/Sanjay/Screws_2024_07_15/img1_43_nosy.jpg 42
43              97.67%
------------------------------------------------------------------------
Average Accuracy: 97.67%
Target: > 95.00%
Status: ✓ PASSED
```

10. Summary Statistics

In [70]:
```
import pandas as pd

df = pd.DataFrame(results)
print("\n=== Detection Summary ===")
print(df)
print(f"\nTotal images processed: {len(df)}")
print(f"Total objects detected: {df['count'].sum()}")
print(f"Average objects per image: {df['count'].mean():.1f}")
```

```
=== Detection Summary ===
                                            filename  count
0  /Users/sanju/Desktop/Sanjay/Screws_2024_07_15/...     42

Total images processed: 1
Total objects detected: 42
Average objects per image: 42.0
```

In [71]:
```
!jupyter nbconvert --to html ai_detection.ipynb
```

```
[NbConvertApp] Converting notebook ai_detection.ipynb to html
[NbConvertApp] Writing 330069 bytes to ai_detection.html
```

Model Information

Segment Anything Model (SAM)

- **Architecture**: Vision Transformer (ViT)
- **Model Size**: ViT-B (~375 MB)
- **Training Data**: SA-1B dataset
  - 11 million images
  - 1.1 billion masks
- **Capabilities**: Zero-shot object segmentation

Training Method

SAM is pre-trained and used in zero-shot mode. No additional training was performed for this task.

Pipeline

1. **Mask Generation**: SAM generates all possible object masks
2. **Area Filtering**: Remove outliers based on median area
3. **Duplicate Removal**: IoU and containment-based filtering
4. **Counting**: Final unique masks = object count

Advantages

✅ Zero-shot generalization
✅ No training required
✅ Handles various object types
✅ High segmentation quality
✅ Robust to lighting/scale variations

Limitations

⚠️ Requires GPU for fast inference
⚠️ Large model size (~375 MB)
⚠️ May oversegment complex textures
⚠️ Sensitive to threshold parameters